

Assignment 1: Intro to World Models and Control

COMP 765 - Fall 2026

Question 1 - Profile of a World Model: DreamerV3

What goes in and what comes out. DreamerV3 is a model-based reinforcement-learning agent rather than a stand-alone video generator. At each environment step it consumes an observation (RGB pixels or a low-dimensional state vector), the previous action, reward, and episode-continuation flag. An encoder maps the observation to categorical stochastic variables z_t ; a recurrent sequence model carries deterministic state h_t . Together they form the compact model state. From it, learned heads reconstruct the observation and predict reward and continuation, while the transition prior predicts the next latent state conditioned on actions. The actor outputs discrete or continuous actions and the critic outputs a distribution over future return [1].

Design principles. The central idea is to learn behavior from imagined latent trajectories, not by rolling pixels forward for every policy update. Replay observations train a recurrent state-space model (RSSM); imagined rollouts of length 16 then train an actor-critic using predicted rewards, continuation probabilities, and bootstrapped lambda-returns. This separates representation learning from decision learning while retaining a differentiable, action-conditioned simulator. Unlike classical system identification, the latent state need not correspond to named physical variables. Unlike MuZero-style task-centric models, Dreamer reconstructs sensory inputs, so its representation is shaped by general observation structure as well as reward. Unlike online search or MPC, the deployed actor selects an action directly without look-ahead search [1].

What distinguishes V3. DreamerV1 introduced analytic actor gradients through continuous latent imagination [2]. DreamerV2 replaced Gaussian stochastic states with multiple categorical variables and used straight-through gradients, reaching human-level Atari performance [3]. DreamerV3 retains discrete RSSM states but targets one fixed configuration across very different domains. Its main robustness devices are: separate stop-gradient KL objectives for dynamics and representation learning; one-nat free bits; a 1% uniform mixture that prevents near-deterministic categoricals; symlog transforms for wide-range signed targets; two-hot categorical reward/return prediction; and percentile-based return normalization. Ablations attribute performance to the combination rather than one trick. The authors report fixed-hyperparameter results on more than 150 tasks spanning Control Suite, Atari, ProcGen, DMLab, Minecraft, and non-visual domains [1].

Assessment. The strongest contribution is robustness: a single agent design handles image and vector observations, sparse and dense rewards, and discrete and continuous actions. Its limitations are also important. Imagined behavior inherits model bias; reconstruction spends capacity on details that may not matter for control; training remains compute-intensive; and benchmark success does not guarantee safe real-world prediction under distribution shift. Public JAX code provides training configurations and reproducible benchmark instructions, but no small public-weight inference model is the core artifact because Dreamer learns online for each environment [4].

Sources for Q1: [1] Hafner et al., Mastering Diverse Domains through World Models, <https://arxiv.org/abs/2301.04104>. [2] Hafner et al., Dream to Control, <https://arxiv.org/abs/1912.01603>. [3] Hafner et al., Mastering Atari with Discrete World Models, <https://arxiv.org/abs/2010.02193>. [4] Official implementation, <https://github.com/danijar/dreamerv3>.

Question 2 - Model and Control the Cart-Pole

A. Linearization about the upright equilibrium

The simulator state is $s = [x, \dot{x}, \theta, \dot{\theta}]^T$. Define the local angle $\phi = \theta - \pi$ and deviation state $\delta = [x, \dot{x}, \theta, \dot{\theta}, \phi]^T$. Near $\theta = \pi$, $\sin(\theta)$ is approximately $-\phi$, $\cos(\theta)$ is approximately -1 , $\cos(\theta)^2$ is approximately 1 , and products such as $\dot{\theta}^2 \sin(\theta)$ are second order and discarded. Let $D = 4(M+m) - 3m$.

$$\begin{aligned} \ddot{x} &= [3m g \phi - 4b \dot{x} + 4u] / D \\ \ddot{\theta} &= [6(M+m)g \phi - 6b \dot{x} + 6u] / (lD) \end{aligned}$$

Therefore $\dot{\delta} = A \delta + B u$, with the state order used by the code:

A =	[0 1 0 0]	B =	[0]
	[0 -4b/D 0 3mg/D]		[4/D]
	[0 -6b/(lD) 0 6(M+m)g/(lD)]		[6/(lD)]
	[0 0 1 0]		[0]

With $M=m=l=0.5$, $g=9.82$, $D=2.5$. The handout states $b=0.1$, giving $A[2,2]=-0.16$ and $A[3,2]=-0.48$. Inspection of the supplied executable CartPole class shows $b=1.0$, so the implemented model uses the following values; all other entries agree:

A =	[0 1.0 0 0]	B =	[0]
	[0 -1.6 0 5.892]		[1.6]
	[0 -4.8 0 47.136]		[4.8]
	[0 0.0 1 0]		[0]

B. LQR design and stability

I used $Q = \text{diag}(2, 1, 2, 120)$ and $R = [0.2]$. The continuous algebraic Riccati equation returns $K = [-3.1623, -6.4776, 7.2892, 47.0906]$. The code applies $u = K(g-s)$, with the angular component wrapped to $[-\pi, \pi]$. Equivalently, $u = -K \delta$. The closed-loop poles are -16.7176 , -7.7261 , and $-0.8903 \pm j 0.6012i$, so the linear model is asymptotically stable. For comparability with swing-up, force is limited to ± 30 N.

Balancing experiments

Each deterministic run used the nonlinear supplied ODE simulator for 20 s at $dt=0.005$ s. Success means that throughout the final 5 s, $|\theta-\pi|<0.10$ rad and $|x|<0.50$ m. The same Q, R, K, and force limit were used for every trial.

initial offset	capture time (s)	max $ u $ (N)	final-window max $ \text{angle} $	result
0.01	0.000	0.47	8.04e-09	PASS
0.10	0.000	4.71	8.09e-08	PASS
$\pi/8$	0.380	18.49	3.51e-07	PASS
$\pi/4$	1.205	30.00	9.88e-07	PASS

All four requested offsets balance and converge essentially to numerical zero. The $\pi/4$ case hits the force limit and takes 1.205 s to meet the capture criterion, whereas smaller errors remain unsaturated. Thus the demonstrated reliable balancing range is at least $|\theta-\pi| \leq \pi/4$ under the stated 30 N limit. This is an empirical region of attraction, not a global-stability claim: the linearization and Riccati proof are local, and saturation makes the nonlinear closed loop piecewise smooth.

C. Swing-up from the downward configuration

With initial-offset= π , the simulator begins at $\theta=2\pi$, physically downward. Pure LQR performs poorly: the local angle error is maximally ambiguous at $\pm\pi$, the command saturates, and the pole does not settle upright during 20 s. Its final-window maximum angle error is 3.1411 rad. Tuning Q and R changes local aggressiveness but does not give LQR a global energy-building strategy.

I therefore explored a hybrid controller. Away from upright it shapes the pole energy

$E = (m l^2/6) \dot{\theta}^2 + (mgl/2)(1-\cos(\theta))$, with target $E^* = mgl$, using $u = -k_E(E^*-E) \text{sign}(\dot{\theta} \cos(\theta)) - k_x x - k_v \dot{x}$, clipped to ± 30 N. I used $k_E=40$, $k_x=1$, and $k_v=2$. When $|\theta-\pi|<0.42$ rad and $|\dot{\theta}|<3.5$ rad/s, control switches to LQR; 0.65 rad hysteresis prevents chatter.

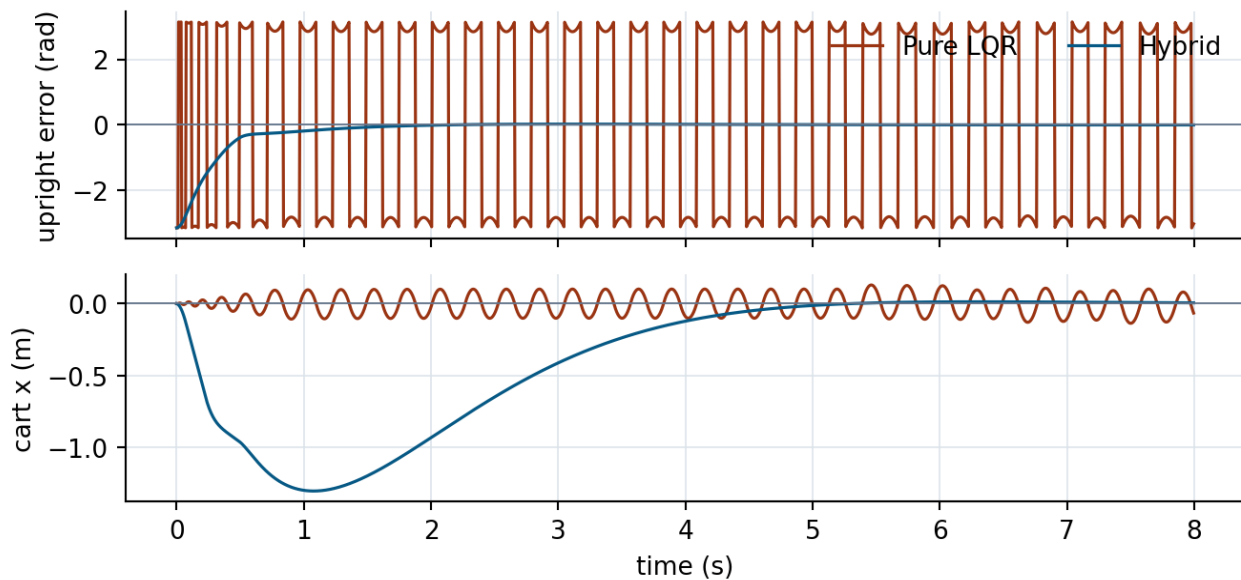


Figure 1. Downward-start comparison. Pure LQR remains near the downward equilibrium; energy shaping builds motion, enters the capture region, and LQR then recenters the cart.

Swing-up result and reproducibility

The hybrid controller first meets $|\theta - \pi| < 0.10$ rad and $|\dot{\theta}| < 0.25$ rad/s at 1.340 s. It passes the 20 s success criterion: over the final 5 s, maximum angle error is $1.14\text{e-}06$ rad and maximum $|x|$ is $4.33\text{e-}06$ m. Peak force is 30 N. This is my best swing-up result. It is stronger than pure LQR but should be interpreted as a deterministic simulation result; actuator delay, sensor noise, and model mismatch were not tested.

Implementation map

`lqr_starter.py` contains the symbolic-to-numeric matrix construction, Q, R, Riccati solve, gain, and pole check. `cartpole_control.py` implements pure LQR plus the optional hybrid swing-up logic. `experiment.py` reruns all six deterministic trials and writes the CSV used for the tables. The original simulator is otherwise unchanged except that its `pygame` import is optional for headless physics tests.

Reproduction

From `assignment1/TeachingCartpole`:

1. `python3 -m pip install -r requirements.txt`
2. `python3 lqr_starter.py`
3. `python3 experiment.py`
4. `python3 cartpole_sim.py --initial-offset 3.141592653589793`

References

- [1] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering Diverse Domains through World Models. arXiv:2301.04104, 2023 (rev. 2024). <https://arxiv.org/abs/2301.04104>
- [2] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. arXiv:1912.01603, 2019. <https://arxiv.org/abs/1912.01603>
- [3] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering Atari with Discrete World Models. arXiv:2010.02193, 2020. <https://arxiv.org/abs/2010.02193>
- [4] D. Hafner et al. DreamerV3 official JAX implementation. <https://github.com/danijar/dreamerv3> (accessed 2026-09-23).
- [5] D. Meger et al. TeachingCartpole simulator. <https://github.com/dmeger/TeachingCartpole> (accessed 2026-09-23).

Scope note: the optional DoubleCartpole bonus was not attempted. All claims in Questions 2B and 2C come from the included executable experiment harness and recorded CSV, not manual observation.